

SOPER 2023/2024

Práctica 2

Antonio Moroño,
Manuel Fernández López,

1. Descripción de los archivos implementados

Aquí explicamos los ficheros adicionales que hemos creado para la realización de la práctica:

main.c: Este es el archivo principal, el cual contiene las funciones más importantes usadas para la elaboración del programa como la función que realizará el proceso principal o la que realizarán los procesos votantes.

handler.c: Este archivo contiene todos los handlers de las señales.

misc.c: Este archivo contiene ciertas funciones que nos facilitan el manejo de las señales.

Makefile: Makefile utilizado para probar el correcto funcionamiento del ejercicio.

2. Descripción de los requisitos

Esta práctica consiste en familiarizarnos con el uso de las señales y los semáforos en C a través de un ejercicio inspirado en uno de los componentes de la tecnología Blockchain. Para ello, hemos elaborado un programa el cual gestiona una selección de un candidato y una votación para aceptarlo o rechazarlo.

Para comunicarse entre los distintos procesos candidato, principal y votante aplicamos el uso de señales (utilizando SIGUSR1 y SIGUSR2).

Además, también tenemos que gestionar las señales SIGALARM y SIGINT, siendo la primera la que marca el fin del programa y la segunda tenemos que manejarla de tal forma que no interrumpa el flujo del programa de golpe, si no que espere a que se realice la última votación previa al envío de la señal antes de terminar.

Por último, hemos utilizado los semáforos para evitar condiciones de carrera y sobreescrituras entre procesos a la hora de plasmar su voto en el archivo dedicado a las votaciones.

3. Descripción de la implementación

Para implementar el funcionamiento pedido en la práctica hemos realizado tres ficheros .c distintos, como bien hemos nombrado previamente, los cuales son **main.c**, **handler.c** y **misc.c** pasaremos a explicar más en profundidad.

El primer archivo que vamos a explicar es **misc.c**. Este archivo contiene una serie de funciones que nos ayudan a manejar correctamente las señales, controlando correctamente el uso de máscaras y configurando las acciones que queremos que realicen nuestros handlers cuando reciban una cierta señal. Aquí podemos encontrar funciones como **suspend_for**, la cual espera a recibir las señales especificadas en los argumentos de la función. También está **config_action**, que configura una estructura sig_action con su handler y sus señales. Por último, tenemos **ignore_signal**, que sirve para ignorar una cierta señal.

El segundo que procedemos a explicar es **handler.c**, el cual contiene todos los handler que hemos creado para controlar el correcto manejo de señales del sistema. Este fichero contiene las funciones handler, para controlar las señales SIGUSR1 y SIGUSR2, handler_ctrlc para controlar la señal SIGINT y handler_alrm para controlar la señal SIGALRM.

Por último, tenemos el fichero **main.c**, aquí es donde el programa principal tiene lugar. Para la creación de los votantes, usamos un bucle en el cual se creen tantos hijos del proceso principal como número de procesos se haya especificado en los argumentos de entrada, y después se envía al proceso principal a la función **principal**, y al resto a la función **votante**.

En la función **principal**, el proceso principal inicia el temporizador con los segundos introducidos en los argumentos de entrada, y envía señales a los procesos votantes para que compitan por ser el proceso candidato. Tras esto queda a la espera de que acabe el temporizador y si se ha recibido la señal SIGINT para acabar el programa.

En la función **votante**, los procesos votantes esperan a que el proceso principal les envíe la señal SIGUSR1 para comenzar a competir por ser el candidato. La forma en que se elige al proceso candidato es a través de un semáforo no bloqueante de forma que el primero que llega al semáforo es el candidato y el resto pasan a ser votantes. Una vez tenemos el proceso candidato, usando sem_wait lo separamos del resto de procesos votantes.

El proceso candidato le envía la señal SIGUSR2 a los procesos votante para que comiencen a votar, después procede a votar y a empezar a comprobar si se han recibido todos los votos. Una vez recibidos todos los votos, muestra el resultado por pantalla y se procede a repetir todo el proceso hasta recibir la señal SIGINT o SIGALRM. Por su parte, los procesos votantes depositan su voto en el fichero usando semáforos para no interrumpirse entre ellos.

Cabe mencionar que para generar los votos hemos usado la función **random_num** la cual tuvimos que desarrollar en otra asignatura. Además de usar esta función, como los procesos votantes al ser todos hijos del proceso principal heredan la misma semilla para **srand** hemos usado los propios pids de los procesos para alterar la semilla dada garantizando la aleatoriedad.

4. Preguntas sobre la práctica

¿Se produce algún problema de concurrencia en el sistema descrito?

Sí, se producen varios problemas de concurrencia ya que los procesos hijos tratan de acceder al mismo archivo de lectura y escritura para almacenar su voto, lo cual puede derivar en que se sobreescriban los votos en el fichero y no se impriman bien.

¿Es sencillo, o siquiera factible, organizar un sistema de este tipo usando únicamente señales?

No, ya que las señales son un mecanismo demasiado simple, rudimentario y poco fiable para gestionar la comunicación entre procesos. Hay mejores alternativas como el uso de la memoria compartida. A gran escala utilizar señales es demasiado complejo y tiene cierta facilidad de que falle. Esto se debe a que puede pasarse por alto el control de ciertas señales que terminen perjudicando el sistema.

5. Pruebas de funcionamiento

Hemos probado distintas entradas de parámetros para probar el sistema, desde parámetros cualquiera, por ejemplo: `./voting 5 10` donde el programa creaba los procesos con normalidad y se realizaban las votaciones hasta que se acabará el tiempo como podemos ver en el ejemplo de ejecución:

```
./voting 5 10
```

```
Candidate 9496 => [ N Y N N Y ] => Rejected
Candidate 9494 => [ N Y Y Y Y ] => Accepted
Candidate 9495 => [ Y N Y Y Y ] => Accepted
Candidate 9496 => [ Y N N Y N ] => Rejected
Candidate 9497 => [ Y Y N N N ] => Rejected
Candidate 9495 => [ N N N Y Y ] => Rejected
Candidate 9496 => [ N N N N N ] => Rejected
Candidate 9495 => [ Y N Y N Y ] => Accepted
Candidate 9496 => [ Y Y N N Y ] => Accepted
Candidate 9497 => [ Y Y N Y N ] => Accepted
Candidate 9496 => [ Y Y Y N Y ] => Accepted
Candidate 9494 => [ Y Y N N Y ] => Accepted
Candidate 9495 => [ Y N N N Y ] => Rejected
Candidate 9494 => [ Y Y Y Y N ] => Accepted
Candidate 9497 => [ Y Y Y N Y ] => Accepted
Candidate 9496 => [ N N Y Y N ] => Rejected
Candidate 9494 => [ Y Y N N N ] => Rejected
Candidate 9495 => [ N Y Y Y Y ] => Accepted
Finishing by alarm
```

Como podemos ver todo funciona con normalidad, tenemos varios procesos candidatos distintos sin que se repita siempre el mismo y los resultados de las votaciones no siguen

ningun patron asegurando la aleatoriedad conseguida con nuestra función **random_num** y con el uso del propio pid del proceso para generar un número más aleatorio.

Si probamos a terminar la ejecución del programa con SIGINT vemos que la señal es manejada sin ningún problema y los procesos terminan ordenadamente:

```
./voting 15 10
Candidate 9704 => [ Y Y Y Y N Y Y Y Y N N Y Y N ] => Accepted
Candidate 9704 => [ Y N Y Y Y Y N N Y Y Y Y Y N Y ] => Accepted
Candidate 9702 => [ Y N Y Y N Y Y N Y Y Y Y Y N N ] => Accepted
Candidate 9708 => [ N N Y Y N N Y Y Y N N Y N N N ] => Rejected
Candidate 9705 => [ Y N N Y Y Y Y N N Y N Y N Y Y ] => Accepted
Candidate 9701 => [ Y N Y Y Y Y Y N Y Y Y N N Y Y ] => Accepted
Candidate 9696 => [ N Y Y N Y Y Y N N Y N N N N Y ] => Rejected
^CFinishing by signal
```

Además sabemos que todos los semáforos se han cerrado correctamente ya que tras finalizar la ejecucion no hay rastro de ellos en el directorio **/dev/shm**.

Si ponemos un caso más extremo como enviar la señal SIGINT justo después de ejecutar el programa vemos que no hay ningún problema:

```
./voting 10 10
^CFinishing by signal
```

Y si ponemos un número de segundos pequeño como por ejemplo 1 vemos que solo da tiempo a realizar entre 1 y 2 votaciones:

```
./voting 20 1
Candidate 10864 => [ Y Y Y Y Y Y Y Y Y N N N Y Y N N Y N N ] => Accepted
Finishing by alarm
```